Booking Flow, Doctor Dashboard and Schedule Audit

Task brief for Claude Code (VS Code) - Vernex Clinic OS - prepared 28 Oct 2026 from a read-only analysis of the current codebase

How to use this document. Save this PDF in the repo as docs/tasks/BOOKING_AND_DOCTOR_FLOW_TASK.pdf, open Claude Code in the project root and paste the starter prompt from Section 0. Claude should work through Tasks 1 to 6 in order and finish each task's acceptance checks before starting the next. Every finding names the real file or function, so no re-discovery is needed, but Claude must still read each file before editing it.

0. Starter prompt (paste this into Claude Code)

```
Read docs/tasks/BOOKING_AND_DOCTOR_FLOW_TASK.pdf completely before changing anything.
It is the full specification for a set of changes to the patient booking flow, the doctor
dashboard and the doctor schedule logic of this repo (React + Vite frontend in frontend/,
Supabase in supabase/).

Work through Tasks 1-6 strictly in order. For each task:
1. Read every file the task names before editing it.
2. Implement exactly what the task says, matching the surrounding code style (services layer in
frontend/src/services, ui components, Tailwind tokens, toast via sonner).
3. Run npm run build and npm run lint inside frontend/ and fix every error.
4. Tick off that task's acceptance checks and tell me what you verified, and how.

Never edit an existing migration file - add new idempotent migrations with a timestamp after
20261013123000.
Never use or ask for the Supabase service-role key; never commit frontend/.env.
Do not push SQL to the remote database yourself: when a task adds a migration, stop and tell me
the exact file, and I will say when to apply it.
If anything in the spec conflicts with the code you find, or a decision is not covered, stop and
ask me instead of guessing.
At the end, give me a short report: what changed, what you tested, what is still open.
```

Decisions already made by the product owner (do not re-open)

- **Bookings stay auto-booked.** A public or WhatsApp booking reserves the slot immediately. There is NO doctor approval step and none must be built. The wording must simply stop implying a doctor approved it.
- **The booking page is the front door.** Opening the site shows the booking page, not the login page. Staff login stays available but is secondary.
- **Patients look up appointments with their mobile number only** (after choosing a clinic). No OTP in this task.

Ground rules for every task

- **Live data only.** Screens read and write through services (frontend/src/services/serviceProvider.ts). No hard-coded sample arrays, names, times, counts or clinic names in pages.
- **The database is the authority.** Anything touching several tables must be a single security definer RPC that is atomic and checks permission server-side. The anonymous (not logged in) path may only use curated RPCs, never read tables directly.
- **Migrations:** new files only. Use create or replace, if not exists, drop policy if exists, on conflict, so they can be re-run.
- **Types:** after changing RPCs or columns, update frontend/src/shared/types/database.types.ts (regenerate with the Supabase CLI if possible, otherwise hand-edit exactly).
- **Dates and time zone:** the clinic works in India time (IST, Asia/Kolkata). Never compute 'today' with new Date().toISOString().slice(0,10) (that is UTC and is wrong from 00:00 to 05:30 IST) and never use current_date in

SQL for a clinic-facing decision (the database runs in UTC). Add ONE shared helper on the frontend (for example localDateISO() in frontend/src/utils/formatDate.ts) and ONE SQL helper that returns the clinic's current date and time, and use them everywhere in these flows. Read the clinic's timezone setting if the clinics table has one; otherwise use a single named constant, Asia/Kolkata, in one place.

- **Mobile:** every new or changed page must work at 375 px width (tables become cards, or scroll horizontally inside their card).
- **Wording:** plain language a patient understands; no jargon like 'RPC' or 'slot id'; every button has a clear label; errors say what to do next.

1. Findings: what is wrong today (confirmed from the code)

#	Finding	Where
A1	Status says CONFIRMED although nobody confirmed. A public booking is created as status 'booked'. The status lookup maps both 'booked' and 'confirmed' to the word 'confirmed'. The success page has fixed text 'Appointment Confirmed - your appointment is confirmed'. There is no confirm action anywhere in the app.	supabase/migrations/20260804120000_phase_15_feature_completion.sql (public_booking_status, about line 766); frontend/src/modules/patient-booking/components/AppointmentSuccessCard.tsx; pages/BookingStatus.tsx; services/supabase/supabaseBooking.service.ts (status mapping); WhatsApp simulator copy
A2	Doctor dashboard hides upcoming bookings. The doctor queue keeps only appointments with status arrived, waiting, in_consultation, completed, no_show. Anything 'booked' (every new public booking) never appears, and there is no way to see tomorrow or later.	frontend/src/modules/doctor/pages/DoctorQueue.tsx (the filter in loadQueue)
A3	Hard-coded values on the doctor dashboard: heading 'Good morning, Doctor' at any hour; clinic name fallback 'Vernex Clinic'; stats followUpsDue, averageWaitingTime and prescriptionsSent are fixed at 0. The same clinic fallback exists on the reception dashboard.	DoctorQueue.tsx; modules/reception/pages/ReceptionDashboard.tsx
A4	Stale slots after a schedule change. Slots are created once per doctor and date and never refreshed. When a doctor edits weekly availability (the service deletes and re-inserts availability rows), already-created slots for future dates stay and patients can still book the old times.	supabaseDoctor.service.ts (updateDoctorAvailability); SQL ensure_doctor_slots uses 'on conflict do nothing'
A5	Wrong 'today' near midnight. Available dates start from current_date in the database (UTC), not IST. Between 00:00 and 05:30 IST patients are offered the previous date. 16 frontend files use the UTC date trick.	SQL public_doctor_available_dates; frontend files using toISOString().slice(0, 10)
A6	Past times are bookable. For today, slots whose start time has already passed are still returned as available.	SQL public_doctor_available_slots
A7	Blocked slots look 'booked'. A slot blocked for a day off is reported to the patient as 'booked'; it should simply not be offered.	SQL public_doctor_available_slots (status case)
A8	Slot generation exists twice, in SQL (ensure_doctor_slots) and in the frontend (getAvailableSlots builds slot rows itself). Two copies can drift apart.	supabaseDoctor.service.ts, getAvailableSlots (about lines 195-255)
A9	Token numbers can collide. next_token_number takes the highest token of the day plus one with no lock, so two patients booking at the same moment can get the same token. Tokens are per clinic per day (A001, A002 ...), not per doctor, so the doctor and date must always be shown next to a token.	SQL next_token_number
A10	The home page goes to login. The route '/' redirects to /login. A booking page already exists at /book (clinic directory), /book/:clinicSlug and /book/:clinicSlug/flow, but visitors never land on it.	frontend/src/routes/AppRoutes.tsx
A11	Status lookup needs a phone AND a booking id or token, and compares the phone exactly as typed (no handling of +91, spaces, leading 0). It returns only ONE appointment (the newest).	SQL public_booking_status; pages/BookingStatus.tsx

2. Task 1 - Honest booking wording and status (no doctor approval)

Why: Finding A1. Patients and doctors must see the same truthful status. A booked slot is reserved, but it has not been 'confirmed by the doctor'.

What to do:

- Define ONE shared set of appointment status labels used by patient pages, reception and doctor screens. Suggested patient wording: booked = 'Booked - your slot is reserved'; arrived or waiting = 'Checked in - waiting'; in_consultation = 'With the doctor'; completed = 'Completed'; cancelled = 'Cancelled'; no_show = 'Missed'. Put the mapping in one place (shared type plus one label helper), not scattered strings.
- Change the status lookup so that 'booked' is returned as 'booked' and is no longer converted to 'confirmed'. Add a new migration (do not edit the old one) that replaces the function. Update BookingStatusResult and the mapping in supabaseBooking.service.ts so 'booked' and the other real statuses pass through unchanged.
- Success page: replace the title and sentence with wording like 'Appointment Booked' and 'Your slot is reserved. Please arrive 10 minutes early and show your token at reception.' Keep the Add to Calendar, Print, WhatsApp and Call buttons.
- Status page: show the label from the shared mapping, never the raw word. Use a positive colour for booked and a distinct, muted colour for cancelled and missed.
- Search every user-facing text for 'confirmed' and 'confirm' (patient booking, WhatsApp booking console and its messages, reception, templates, notification previews). Keep the word only on a button that asks the patient to submit ('Confirm booking' before it is saved is fine) and change any text that claims a doctor or clinic approved it.
- Do NOT add a doctor Confirm or Decline button, a 'requested' flow or any approval table.

Acceptance checks:

- Create a public booking. The success page and the status page both say Booked, and neither says confirmed by the doctor.
- Mark the same appointment arrived from reception: the patient status page changes to Checked in. Complete it: it changes to Completed. Cancel another one: it shows Cancelled.
- The WhatsApp booking console messages and the reception previews use the same wording.
- npm run build and npm run lint pass; the new migration file is named and handed to me, not applied.

3. Task 2 - Doctor dashboard: same status, and an easy view of upcoming appointments

Why: Findings A2 and A3. The doctor cannot currently see new bookings at all, and parts of the page are fixed text.

What to do:

- Rework the doctor landing page (route /doctor/queue, file DoctorQueue.tsx) into two clear parts, using tabs or two stacked sections that work on a phone: **Today** and **Upcoming**.
- **Today**: ALL of the doctor's appointments for today (India date), sorted by time, including booked ones. Each row or card shows: token, time, patient name, age or gender if known, reason or main problem, source (walk-in, phone, QR, WhatsApp, website), and the status label from Task 1. Keep the live queue (arrived, waiting, in consultation) visible at the top. Booked patients can be opened to view the patient, but a consultation still starts only for patients who are checked in (keep the existing rule).
- **Upcoming**: tomorrow and the next 7 days by default, grouped under date headings (for example 'Thu, 29 Oct - 6 appointments'), each group sorted by time. Add a range filter (7 days, 30 days, custom date) and a search by patient name or phone. Show a count on each tab. Empty states in plain words, for example 'No appointments in the next 7 days.'

- Add a small summary row with real numbers from the database: today total, checked in, waiting, completed today, upcoming this week. Compute follow-ups due, average waiting time and prescriptions sent from real data, or remove a tile if it cannot be computed honestly. No fixed zeros.
- Replace 'Good morning, Doctor' with a greeting based on the current IST hour and the doctor's real name (for example 'Good afternoon, Dr. Aisha Khan'). Remove the 'Vernex Clinic' fallback: if the clinic name is missing, show nothing rather than a made-up name.
- Fetch only what is needed: add a service method that queries appointments by doctor, date range and optional search in the database, instead of loading every appointment and filtering in the browser. Load patient details in one query, not one request per patient.
- Refresh: keep the Refresh button and also refresh automatically when new bookings arrive (realtime subscription or a 30 second poll, as the pharmacy queue does). Show a small 'Updated just now' text.
- A booking made on the public page must appear in the doctor's Today or Upcoming list within the refresh interval, with the same token the patient received.

Acceptance checks:

- Book a public appointment for the doctor for tomorrow: it appears under Upcoming with the right time, token and patient, and the status wording matches what the patient sees.
- Book one for today: it appears under Today even before check-in.
- At 375 px width both tabs are usable without the whole page scrolling sideways.
- Search the doctor dashboard code and confirm no literal clinic names, no greeting without a time check, and no fixed zeros are left.

4. Task 3 - Schedule audit: are the times shown exactly what the doctor set?

Why: Findings A4 to A8. The product owner asked whether the schedule times shown to patients match the doctor's availability. Today they can be wrong. Fix it and prove it.

What to do:

- **Single source of truth.** Keep slot creation only in the database function ensure_doctor_slots. Make the frontend (getAvailableSlots and anything else that inserts slot rows) call the database function instead of building slots itself, so there is one algorithm.
- **Honour every setting:** weekly working hours per day, break start and end, slot length (slot_duration_minutes), patients per slot (max_appointments_per_slot), blocked dates, and doctor active status. A slot must never start in a break, never run past the end time, and a disabled day must offer nothing.
- **Edits must take effect.** When the doctor saves new availability, blocks a date, or changes slot length or capacity, future slots for that doctor that have no booking must be removed or regenerated so patients see the new times. Slots that already have a booking must be kept, and the doctor must be told in plain words which bookings fall outside the new hours so reception can move them. Never silently delete booked slots. Put this logic in one SQL function called by the save action, in one transaction.
- **India time.** The first available date and 'today' must use the clinic's local date (see ground rules). For today, hide slots whose start time has already passed (allow a lead time, default 0 minutes, defined in one place).
- **Blocked slots** must not be returned to patients at all; only genuinely full slots may be shown as 'Full'.
- **Display.** Times are shown in 12 hour format with AM or PM everywhere (patient booking, status page, reception, doctor). A slot of 09:00 saved by the doctor must show as 09:00 AM on every screen, not shifted. Check the doctor Availability page too: a disabled day must not save a default time such as 09:00 by accident.
- **Write a short audit note** in docs/SCHEDULE_AUDIT.md with a table: doctor setting, where it is applied, tested yes or no, result. Include the test scenario below with the actual outcomes.

Acceptance checks:

- Scenario to run and report: doctor works Mon-Fri 09:00-13:00 with a break 11:00-11:30 and 15 minute slots. The patient page offers 09:00 to 10:45, then 11:30 to 12:45, and nothing from 11:00 to 11:30, and nothing on Saturday and Sunday.
- Change the doctor to 10:00-12:00 on Monday. Without any manual step, the patient page now offers only 10:00-11:45 on future Mondays. A future Monday that already had a 09:00 booking keeps that booking and the doctor sees a warning about it.
- Block a date: it disappears from the patient date list, and no slot shows as 'booked' for it.
- Change the slot length to 30 minutes: future empty slots are regenerated at 30 minutes.
- Simulate 01:00 IST: the first offered date is today in India. At 10:20 IST, today's 09:00 and 10:15 slots are not offered.
- Two people trying to book the last slot at the same time: exactly one succeeds, the other gets a clear 'This slot was just taken, please pick another' message.

5. Task 4 - No hard-coded data in the booking workflow, and end-to-end proof

Why: The product owner wants the whole workflow to run on real data. Findings A3, A5 and A9 are the known cases; this task finds the rest.

What to do:

- Audit these areas line by line: public booking (all pages under modules/patient-booking), WhatsApp booking (modules/whatsapp-booking and services/supabase/supabaseWhatsApp.service.ts), reception (dashboard, appointments, queue, walk-in) and doctor (queue, availability, consultation start). List every literal that stands in for data: clinic or doctor names, times, tokens, prices, counts, greeting text, default fees, a fixed payment status (the appointment mapper sets paymentStatus to 'pending' for everyone), and placeholder preview panels.
- For each item either connect it to real data or, if it is deliberately a labelled demo preview (for example the reception WhatsApp 'Hi' preview card that shows token A014), keep it only if its label makes clear it is an example; otherwise remove it. Record each decision in the audit note.
- Replace every UTC date trick in appointment, doctor, reception and booking code with the shared IST helper (16 files use the pattern; pharmacy and billing files should be fixed too wherever they show 'today').
- Token numbers: make token creation safe under concurrency (lock per clinic and date, or an atomic counter) and add a unique rule so the same token cannot exist twice for one clinic and date. Keep the existing format (A001, A002 ...). Decide and handle what happens past A999. Do not change existing tokens.
- Run the full workflow by hand against the real database and write the results into docs/BOOKING_WORKFLOW_TEST.md: (1) public booking, (2) the same appointment visible on reception, (3) visible on the doctor's Today or Upcoming, (4) check-in by reception, (5) consultation by the doctor, (6) status page updated at each step, (7) cancel and re-book the freed slot, (8) a WhatsApp-console booking follows the same path. Include the token and status seen at each step.

Acceptance checks:

- The audit note lists every literal found and what was done with it. A final search for the fallback strings ('Vernex Clinic', 'Good morning, Doctor', fixed zeros in stats) in the touched files returns nothing.
- Two bookings created at the same instant get different tokens; a duplicate token insert is rejected by the database.
- The workflow test document shows all 8 steps passed, with real values.

6. Task 5 - The booking page comes first

Why: Finding A10. Visitors should land on a booking page, not on staff login.

What to do:

- Change the route '/' so it shows a public landing page instead of redirecting to /login. Keep /login working exactly as now.

- Landing page content: the product or clinic network name, one short sentence, a large primary button **Book an appointment**, a secondary button **My appointments** (opens the lookup from Task 6), and a small, quiet **Staff login** link in the corner. Nothing else is required. It must look good at 375 px.
- Book an appointment opens the clinic list (the existing /book directory, services.booking.getPublishedClinics). The patient picks a clinic, then continues through the existing steps: doctor, service, date, time, patient details, booked. Do not rebuild these steps; reuse them and only fix what Tasks 1 and 3 require. If only one clinic is published, skip straight to that clinic.
- A staff member who is already signed in and opens '/' should be sent to their own dashboard (the same role-based home they get after login), without flashing the landing page first. A visitor with no session sees the landing page.
- The login page keeps its 'Book an appointment' link, and the logo on public pages links to '/'. Give every public page a clear browser tab title.

Acceptance checks:

- Open the site root in a private window: the landing page shows, not login. Click Book an appointment: clinics are listed. Click Staff login: the login page appears.
- Sign in as a doctor, then open '/': you land on the doctor page directly.
- No console errors; works at phone width.

7. Task 6 - Find my appointments with only a mobile number (after choosing a clinic)

Why: Finding A11. Patients pick their clinic, type their mobile number, and see all their appointments, each with the token number.

What to do:

- Flow: landing page or success page > **My appointments** > choose clinic (the same clinic list as booking; preselect it if there is only one) > enter mobile number > **Show my appointments**.
- Replace the current two-field page (frontend/src/modules/patient-booking/pages/BookingStatus.tsx, route /booking/status) with this flow. Keep the route working.
- Result: a list of cards. Upcoming appointments first (soonest first), then past ones (newest first), limited to the last 90 days of history. Each card shows: **token number (large)**, date, time in 12 hour format, doctor name and specialty, clinic name and address, patient name, the status label from Task 1, the service if any, and the booking id (small). Add the line 'Show your token number at reception.' Cancelled and missed appointments are shown but visually muted.
- Provide a new RPC (new migration) that takes the clinic slug and the phone number and returns this list. It is the ONLY thing the anonymous page may call for this feature. It must: match the clinic by slug and return only appointments of that clinic; normalise phone numbers before comparing (ignore spaces and dashes, a leading +91, 91 or 0, and compare the last 10 digits; use the same normalisation when bookings save the number, and make sure older records still match); return only the fields listed above and NEVER clinical data (no diagnosis, notes, prescriptions, vitals, billing, or other patients' data); cap the number of rows; return an empty list (not an error) when nothing matches.
- **Abuse protection (required, because a mobile number alone is a weak check):** add server-side throttling for this RPC (for example a small table that counts lookups per phone and per clinic in a rolling window and refuses after a limit with a friendly 'Too many tries, please try again in a few minutes'). Mask the patient's name partially in the result (first name plus initials of the rest) so a stranger who guesses a number learns as little as possible. Do not add OTP now, but note in the docs that OTP by SMS or WhatsApp is the recommended next step once a provider is connected, and state this risk plainly in your final report.
- Empty state in plain words ('We could not find any appointments for this number at this clinic. Check the number or the clinic.'), plus loading and error states. A 'Book a new appointment' button under the list. Usable at 375 px.

- The booking success page gets a link 'View all my appointments' that opens this flow with the clinic and number already filled.
- Remove or repurpose the old status lookup RPC only if nothing else uses it; if it is kept, apply the same phone normalisation.

Acceptance checks:

- Create 2 bookings for one number, one for tomorrow and one in the past (use demo data or a manual insert). Enter that number: both show, upcoming first, each with its token, doctor, date, time and status.
- Enter the same number with +91, with spaces, and with a leading 0: same result.
- Choose a different clinic: nothing from the first clinic appears.
- A wrong number returns the friendly empty state; after the allowed number of quick attempts the 'too many tries' message appears.
- Inspect the response in the browser network tab: it contains none of the forbidden clinical or billing fields.

8. Final verification and report

1 npm run build and npm run lint pass in frontend/ with no new warnings in touched files.

2 List every new migration file in order, each idempotent, none edited from before, and the exact steps I should use to apply them. Do not apply them yourself.

3 docs/SCHEDULE_AUDIT.md and docs/BOOKING_WORKFLOW_TEST.md exist and contain real results, not templates.

4 Update README.md (landing page, public booking, 'My appointments', status wording) and the relevant phase docs so they match the new behaviour.

5 Final message to me: a short list of what changed per task, what you tested and how, anything you could not test, and any decision you need from me. State the mobile-number-only risk plainly.

Out of scope (do not do): a doctor approval or decline flow, OTP or SMS sending, payment changes, pharmacy or billing redesign, new roles, and any change to RLS beyond what these tasks need.